

Endgame-13 Sprint-4 Problem Solving Task

Rules & Requirements:

File Naming & Structure:

- Create a single public GitHub repository.
- All solutions must be contained within a single file named exactly: `EG-13_Sprint-4_PS.js`
- Do not create separate files for individual problems. Submitting multiple files will result in a point deduction.

Function Signatures & Boilerplates:

- Use the exact function names, parameters, and boilerplate templates specified in this document. Modifying function names or parameters will result in test failure and mark deductions.

Submission Deadline:

- Submissions submitted after the deadline will not be accepted or evaluated.

Submission Deadline:

Sprint-4 Problem Solving	Date & Time
Task Release	Septembre 10, 2026
Submission Deadline	August 13, 2026 at 5:30 PM

Submission Process:

1. Commit your finalized `EG-13_Sprint-4_PS.js` file to your GitHub repository and push to the default `main` branch.
2. Ensure your repository visibility is set to Public so evaluators can review your work.
3. Copy your repository URL (e.g., <https://github.com/your-username/your-repo-name>) and submit the Google form before the deadline.
4. Submission Form: <https://forms.gle/1R5g23Be7ja8qFu46>

01. Isomorphic Strings

Write a validation function that determines whether two strings are isomorphic. Each character in the first string must map to exactly one character in the second string, while preserving the order of characters.

```
/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isIsomorphic = function(s, t) {

};
```

Expected Input: "egg", "add"

Expected Output: true

02. Word Pattern

Write a validation function that determines whether a string follows a given word pattern. Each pattern character must map to exactly one word, and each word must map to exactly one pattern character.

```
/**
 * @param {string} pattern
 * @param {string} s
 * @return {boolean}
 */
var wordPattern = function(pattern, s) {

};
```

Expected Input: "abba", "dog cat cat dog"

Expected Output: true

03. Find the Difference

Write a function that finds the extra character added to string **t** when string **s** is rearranged and one additional character is added.

```
/**
 * @param {string} s
 * @param {string} t
 * @return {character}
 */
var findTheDifference = function(s, t) {

};
```

Expected Input: "abcd", "abcde"

Expected Output: "e"

04. Reverse Linked List

Write a transformation function that reverses a singly linked list and returns the new head of the reversed list.

```
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var reverseList = function(head) {

};
```

Expected Input: [1, 2, 3, 4, 5]

Expected Output: [5, 4, 3, 2, 1]

05. Middle of the Linked List

Write a function that finds and returns the middle node of a singly linked list. If the list contains two middle nodes, return the second middle node.

```
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var middleNode = function(head) {

};
```

Expected Input: [1, 2, 3, 4, 5]

Expected Output: [3, 4, 5]

06. Product of Array Except Self

Write a function that returns an array where each element is the product of all elements in the input array except the element at the current index. Do not use division.

```
/**
 * @param {number[]} nums
 * @return {number[]}
 */
var productExceptSelf = function(nums) {

};
```

Expected Input: [1, 2, 3, 4]

Expected Output: [24, 12, 8, 6]

07. Remove Nth Node From End of List

Write a function that removes the `n`th node from the end of a singly linked list and returns the head of the modified list.

```
/**
 * @param {ListNode} head
 * @param {number} n
 * @return {ListNode}
 */
var removeNthFromEnd = function(head, n) {

};
```

Expected Input: [1, 2, 3, 4, 5], n = 2

Expected Output: [1, 2, 3, 5]

08. Find First and Last Position of Element in Sorted Array

Write a function that finds the starting and ending position of a given target value in a sorted array. Return [-1, -1] if the target does not exist.

```
/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
var searchRange = function(nums, target) {

};
```

Expected Input: [5, 7, 7, 8, 8, 10], target = 8

Expected Output: [3, 4]

09. Permutation in String

Write a validation function that determines whether one string contains a permutation of another string as a substring.

```
/**
 * @param {string} s1
 * @param {string} s2
 * @return {boolean}
 */
var checkInclusion = function(s1, s2) {

};
```

Expected Input: "ab", "eidbaooo"

Expected Output: true

10. Find All Anagrams in a String

Write a function that finds all starting indices of substrings in **s** that are anagrams of string **p**. Return the indices in any order.

```
/**
 * @param {string} s
 * @param {string} p
 * @return {number[]}
 */
var findAnagrams = function(s, p) {

};
```

Expected Input: "cbaebabacd", "abc"

Expected Output: [0, 6]